MediFlow — Backend Developer Documentation

Version: 2.0 | **Date:** March 16, 2026
Purpose: Complete backend specification for a Django + PostgreSQL API that powers the MediFlow frontend
Frontend Repo: https://github.com/prasadpanvalkar/mediflow.git

1. WHAT IS MEDIFLOW

MediFlow is a modern Indian Pharmacy Management SaaS built to replace Marg ERP 9+ Gold. It is a multi-outlet, multi-staff pharmacy operations platform covering billing, inventory, purchases, customer management, staff attendance, and accounts.
Core design principle:
The frontend is 100% complete with mock data. The backend's only job is to implement the API contract defined in this document. When the real API is connected, zero frontend code changes — only the USE_MOCK env flag flips to false.
Target user: Pharmacy counter staff in India. Must work on a ₹8,000 Android phone in a Chrome browser.

2. SYSTEM ARCHITECTURE

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                    CLIENT (Browser / Mobile)                                 │
│          Next.js 14 — App Router — TypeScript                                │
│          shadcn/ui + Tailwind CSS                                            │
│          Zustand (client state) + TanStack Query (server)                    │
└──────────────────────────────────────────────────────────────────────────────┘
                    │ HTTPS / REST JSON
                    ▼
┌──────────────────────────────────────────────────────────────────────────────┐
│                    Nginx Reverse Proxy                                       │
│   /api/* → Django    /  → Next.js    /media/* → Files                        │
└──────────────────────────────────────────────────────────────────────────────┘
        │                    │
        ▼                    ▼
┌──────────────────────┐   ┌──────────────────────────┐
│  Django 4.2          │   │  Next.js Server          │
```

```
│ Django REST   │      │ (SSR pages)      │
│ Framework     │      └──────────────────┘
│ Simple JWT    │
└───────────────┴──────┘
        │
        ├──► PostgreSQL 15  (primary database)
        ├──► Redis 7        (caching, sessions)
        └──► Cloudflare R2  (file storage: invoices, photos)
```

Docker Compose services: frontend, backend, db, redis, nginx — all in one docker-compose up.

3. TECH STACK

Layer	Technology	Version
Backend Framework	Django	4.2
API Layer	Django REST Framework	3.14+
Authentication	Simple JWT	latest
Database	PostgreSQL	15
Cache / Sessions	Redis	7
File Storage	Cloudflare R2 (S3-compatible)	—
Background Tasks	Celery + Redis	(Phase 2)
Containerization	Docker + Docker Compose	—
Web Server	Nginx	latest

4. MULTI-TENANCY MODEL

MediFlow is **outlet-based multi-tenant**. Every single database record is scoped to an outletId.

```
Organization (1)
  └── Outlet (many)        ← one pharmacy branch = one outlet
      └── Staff        ← staff belong to one outlet
      └── Products     ← product catalog per outlet
      └── Batches      ← stock per outlet
      └── Customers    ← customer base per outlet
      └── All transactions
```

Rule for every API endpoint: Always filter by outletId from the authenticated user's JWT claims. Never return data across outlets.

5. AUTHENTICATION

- **JWT tokens** via Simple JWT
- Token stored in httpOnly cookie (not localStorage — security)

- Every request carries Authorization: Bearer <token>
- JWT payload contains: userId, outletId, role, name

Staff Roles

```
STAFF_ROLES = [
   'super_admin',   # full access, can manage outlets
   'admin',         # full access within outlet
   'manager',       # all ops, no staff management
   'billing_staff', # billing + inventory view only
   'view_only',     # read-only reports
]
```

Auth Endpoints

```
POST /api/auth/login/        → { access, refresh, user: { id, name, role, outletId } }
POST /api/auth/refresh/      → { access }
POST /api/auth/logout/
GET  /api/auth/me/           → current user profile
```

6. COMPLETE DATABASE SCHEMA

6.1 Core Tables

```
-- ── Outlet ──────────────────────────────────────────────────────────────
Outlet (
   id        UUID PK,
   name      VARCHAR(200),
   address   TEXT,
   city      VARCHAR(100),
   state     VARCHAR(100),
   gstin     VARCHAR(15),
   drugLicenseNo VARCHAR(50),
   phone     VARCHAR(20),
   email     VARCHAR(200),
   createdAt TIMESTAMP
)

-- ── Staff ───────────────────────────────────────────────────────────────
Staff (
   id        UUID PK,
```

```
  outletId   UUID FK → Outlet,
  name       VARCHAR(200),
  role       VARCHAR(20),       -- StaffRole enum
  phone      VARCHAR(20),
  email      VARCHAR(200),
  pin        VARCHAR(6),        -- hashed, for billing PIN entry
  isActive   BOOLEAN DEFAULT true,
  joinDate   DATE,
  salary     DECIMAL(10,2),
  photoUrl   VARCHAR(500),      -- Cloudflare R2
  createdAt  TIMESTAMP
)

-- ── Product (Master Catalog) ──────────────────────────────────────
Product (
  id            UUID PK,
  name          VARCHAR(200),
  genericName   VARCHAR(200),   -- salt/composition
  manufacturer  VARCHAR(200),
  hsn           VARCHAR(10),
  schedule      VARCHAR(20),    -- 'OTC'|'H'|'H1'|'X'|'Narcotic'
  drugType      VARCHAR(20),    -- 'allopathy'|'ayurveda'|'homeo'|'fmcg'
  gstRate       DECIMAL(5,2),
  cessRate      DECIMAL(5,2) DEFAULT 0,
  isNarcotic    BOOLEAN DEFAULT false,
  createdAt     TIMESTAMP
)

-- ── OutletProduct (per-outlet product settings) ──────────────────
OutletProduct (
  id            UUID PK,
  outletId      UUID FK → Outlet,
  productId     UUID FK → Product,
  customName    VARCHAR(200),   -- outlet's preferred display name
  mrp           DECIMAL(10,2),
  saleRate      DECIMAL(10,2),
  reorderQty    INTEGER DEFAULT 0,
  minQty        INTEGER DEFAULT 0,
  maxQty        INTEGER DEFAULT 0,
  rackLocation  VARCHAR(50),
  isActive      BOOLEAN DEFAULT true,
  UNIQUE(outletId, productId)
)
```

```
-- ── Batch (THE inventory table) ──────────────────────────────────────
Batch (
  id            UUID PK,
  outletId      UUID FK → Outlet,
  productId     UUID FK → Product,
  purchaseItemId  UUID FK → PurchaseItem,   -- traceability
  batchNo       VARCHAR(100),
  mfgDate       DATE,
  expiryDate    DATE,                  -- NOT NULL
  qtyStrips     INTEGER DEFAULT 0,       -- current stock
  qtyLoose      INTEGER DEFAULT 0,       -- loose tablets
  mrp           DECIMAL(10,2),
  purchaseRate  DECIMAL(10,2),
  ptr           DECIMAL(10,2),          -- price to retailer
  pts           DECIMAL(10,2),          -- price to stockist
  godown        VARCHAR(100) DEFAULT 'main',
  isActive      BOOLEAN DEFAULT true,
  createdAt     TIMESTAMP,
  INDEX(outletId, productId, expiryDate)   -- FEFO queries
)
```

6.2 Distributor & Purchase Tables

```
-- ── Distributor ──────────────────────────────────────────────────────
Distributor (
  id            UUID PK,
  outletId      UUID FK → Outlet,
  name          VARCHAR(200),
  gstin         VARCHAR(15),
  drugLicenseNo  VARCHAR(50),
  phone         VARCHAR(20),
  email         VARCHAR(200),
  address       TEXT,
  city          VARCHAR(100),
  state         VARCHAR(100),
  creditDays    INTEGER DEFAULT 30,
  openingBalance  DECIMAL(12,2) DEFAULT 0,
  balancingMethod VARCHAR(20) DEFAULT 'bill_by_bill',  -- 'bill_by_bill'|'fifo'|'on_account'
  isActive      BOOLEAN DEFAULT true,
  createdAt     TIMESTAMP
)
```

```
-- ── PurchaseInvoice (GRN header) ──────────────────────────────────────
PurchaseInvoice (
  id            UUID PK,
  outletId      UUID FK → Outlet,
  distributorId  UUID FK → Distributor,
  purchaseType   VARCHAR(10),   -- 'cash' | 'credit'
  invoiceNo      VARCHAR(100),
  invoiceDate    DATE,
  dueDate        DATE,
  subtotal       DECIMAL(12,2),
  discountAmount  DECIMAL(12,2) DEFAULT 0,
  gstAmount      DECIMAL(12,2) DEFAULT 0,
  cessAmount     DECIMAL(12,2) DEFAULT 0,
  freight        DECIMAL(10,2) DEFAULT 0,
  roundOff       DECIMAL(5,2) DEFAULT 0,
  grandTotal     DECIMAL(12,2),
  amountPaid     DECIMAL(12,2) DEFAULT 0,
  outstanding    DECIMAL(12,2),  -- grandTotal - amountPaid (computed + stored)
  godown         VARCHAR(100) DEFAULT 'main',
  notes          TEXT,
  invoiceImageUrl VARCHAR(500),  -- scanned invoice photo
  createdBy      UUID FK → Staff,
  createdByName  VARCHAR(200),
  createdAt      TIMESTAMP
)

-- ── PurchaseItem (GRN line items) ─────────────────────────────────────
PurchaseItem (
  id            UUID PK,
  purchaseInvoiceId UUID FK → PurchaseInvoice,
  productId      UUID FK → Product,
  isCustomProduct BOOLEAN DEFAULT false,  -- user-typed name, no master record
  customProductName VARCHAR(200),
  batchNo        VARCHAR(100),
  expiryDate     DATE,
  qty            INTEGER,
  freeQty        INTEGER DEFAULT 0,     -- scheme quantity
  purchaseRate   DECIMAL(10,2),         -- excl. GST
  ptr            DECIMAL(10,2),
  pts            DECIMAL(10,2),
  mrp            DECIMAL(10,2),
  gstRate        DECIMAL(5,2),
  cessRate       DECIMAL(5,2) DEFAULT 0,
```

```
  cessAmount     DECIMAL(10,2) DEFAULT 0,
  discountPct    DECIMAL(5,2) DEFAULT 0,
  totalAmount    DECIMAL(12,2),
  createdAt      TIMESTAMP
)
```

6.3 Sales Tables

```
-- ── Customer ──────────────────────────────────────────────────────────────
Customer (
  id            UUID PK,
  outletId      UUID FK → Outlet,
  name          VARCHAR(200),
  phone         VARCHAR(20),
  email         VARCHAR(200),
  address       TEXT,
  dateOfBirth   DATE,
  bloodGroup    VARCHAR(5),
  allergies     TEXT[],       -- PostgreSQL array
  chronicConditions TEXT[],
  doctorName    VARCHAR(200),
  creditLimit   DECIMAL(10,2) DEFAULT 0,
  outstandingBalance DECIMAL(12,2) DEFAULT 0,
  totalPurchases  DECIMAL(14,2) DEFAULT 0,
  isActive      BOOLEAN DEFAULT true,
  createdAt     TIMESTAMP
)

-- ── SaleInvoice ──────────────────────────────────────────────────────────
SaleInvoice (
  id            UUID PK,
  outletId      UUID FK → Outlet,
  invoiceNo     VARCHAR(100),  -- auto-generated: INV-2026-001234
  customerId    UUID FK → Customer NULLABLE,
  customerName  VARCHAR(200),  -- denormalized for walk-ins
  billDate      DATE,
  paymentMode   VARCHAR(20),   -- 'cash'|'upi'|'card'|'credit'|'split'
  subtotal      DECIMAL(12,2),
  discountAmount  DECIMAL(12,2) DEFAULT 0,
  gstAmount     DECIMAL(12,2) DEFAULT 0,
  roundOff      DECIMAL(5,2) DEFAULT 0,
```

```
  grandTotal     DECIMAL(12,2),
  amountPaid     DECIMAL(12,2),
  outstanding    DECIMAL(12,2) DEFAULT 0,
  cashAmount     DECIMAL(10,2) DEFAULT 0,   -- for split payment
  upiAmount      DECIMAL(10,2) DEFAULT 0,
  cardAmount     DECIMAL(10,2) DEFAULT 0,
  upiRef         VARCHAR(100),
  requiresPrescription BOOLEAN DEFAULT false,
  prescriptionRef VARCHAR(200),
  createdBy      UUID FK → Staff,
  createdByName  VARCHAR(200),
  createdAt      TIMESTAMP
)

-- ── SaleItem ──────────────────────────────────────────────────────────────
SaleItem (
  id             UUID PK,
  saleInvoiceId  UUID FK → SaleInvoice,
  productId      UUID FK → Product,
  batchId        UUID FK → Batch,       -- FEFO selected batch
  batchNo        VARCHAR(100),          -- denormalized
  expiryDate     DATE,                  -- denormalized
  qty            INTEGER,
  mrp            DECIMAL(10,2),
  saleRate       DECIMAL(10,2),
  gstRate        DECIMAL(5,2),
  discountPct    DECIMAL(5,2) DEFAULT 0,
  totalAmount    DECIMAL(12,2)
)

-- ── ScheduleHRegister ─────────────────────────────────────────────────────
-- Legal requirement for Schedule H/H1/Narcotic drugs
ScheduleHRegister (
  id             UUID PK,
  outletId       UUID FK → Outlet,
  saleInvoiceId  UUID FK → SaleInvoice,
  saleItemId     UUID FK → SaleItem,
  productId      UUID FK → Product,
  productName    VARCHAR(200),
  schedule       VARCHAR(20),
  qty            INTEGER,
  doctorName     VARCHAR(200),
```

```
  patientName    VARCHAR(200),
  patientAge     INTEGER,
  prescriptionNo  VARCHAR(100),
  billDate       DATE,
  createdAt      TIMESTAMP
)
```

6.4 Accounts Tables

```
-- ── PaymentEntry (you pay distributor) ──────────────────────────────
PaymentEntry (
  id            UUID PK,
  outletId       UUID FK → Outlet,
  distributorId   UUID FK → Distributor,
  date           DATE,
  totalAmount    DECIMAL(12,2),
  paymentMode    VARCHAR(20),   -- 'cash'|'upi'|'cheque'|'bank_transfer'
  referenceNo    VARCHAR(100),  -- UTR / cheque no / transaction ID
  notes          TEXT,
  createdBy      UUID FK → Staff,
  createdAt      TIMESTAMP
)

-- ── PaymentAllocation (bill-by-bill linking) ────────────────────────
PaymentAllocation (
  id            UUID PK,
  paymentEntryId  UUID FK → PaymentEntry,
  purchaseInvoiceId UUID FK → PurchaseInvoice,
  allocatedAmount DECIMAL(12,2)
)

-- ── ReceiptEntry (customer pays you) ────────────────────────────────
ReceiptEntry (
  id            UUID PK,
  outletId       UUID FK → Outlet,
  customerId     UUID FK → Customer,
  date           DATE,
  totalAmount    DECIMAL(12,2),
  paymentMode    VARCHAR(20),
  referenceNo    VARCHAR(100),
  notes          TEXT,
  createdBy      UUID FK → Staff,
  createdAt      TIMESTAMP
```

```
)

-- ── ReceiptAllocation ──────────────────────────────────────────────────────
ReceiptAllocation (
  id            UUID PK,
  receiptEntryId  UUID FK → ReceiptEntry,
  saleInvoiceId   UUID FK → SaleInvoice,
  allocatedAmount DECIMAL(12,2)
)

-- ── ExpenseEntry
──────────────────────────────────────────────────────────────────────────
ExpenseEntry (
  id            UUID PK,
  outletId       UUID FK → Outlet,
  date           DATE,
  expenseHead     VARCHAR(50),    --
'rent'|'salary'|'electricity'|'transport'|'maintenance'|'marketing'|'other'
  customHead      VARCHAR(100),   -- when expenseHead = 'other'
  amount          DECIMAL(10,2),
  paymentMode     VARCHAR(20),
  referenceNo     VARCHAR(100),
  notes           TEXT,
  createdBy       UUID FK → Staff,
  createdAt       TIMESTAMP
)

-- ── LedgerEntry (auto-generated, never manually inserted) ──────────
LedgerEntry (
  id            UUID PK,
  outletId       UUID FK → Outlet,
  entityType      VARCHAR(20),    -- 'distributor'|'customer'|'cash'|'bank'
  entityId        UUID,           -- distributorId or customerId
  date           DATE,
  entryType       VARCHAR(30),    --
'purchase'|'payment'|'sale'|'receipt'|'expense'|'opening_balance'
  referenceId     UUID,           -- FK to source record
  referenceNo     VARCHAR(100),   -- invoice no or payment ref
  description     TEXT,
  debit           DECIMAL(12,2) DEFAULT 0,
  credit          DECIMAL(12,2) DEFAULT 0,
  balance         DECIMAL(12,2),  -- running balance
  createdAt       TIMESTAMP,
```

```
  INDEX(outletId, entityType, entityId, date)
)
```

6.5 Attendance Tables

```
-- ── AttendanceRecord ──────────────────────────────────────────────────────
AttendanceRecord (
  id            UUID PK,
  outletId      UUID FK → Outlet,
  staffId       UUID FK → Staff,
  date          DATE,
  checkInTime     TIMESTAMP,
  checkOutTime    TIMESTAMP,
  checkInPhotoUrl VARCHAR(500),   -- Cloudflare R2
  checkOutPhotoUrl VARCHAR(500),
  status        VARCHAR(20),    -- 'present'|'absent'|'half_day'|'late'
  gracePeriodUsed BOOLEAN DEFAULT false,
  notes         TEXT,
  UNIQUE(staffId, date)
)
```

7. COMPLETE API SPECIFICATION

Base URL: /api/v1/
All endpoints require: Authorization: Bearer <token>
All responses follow:

```
{
  "success": true,
  "data": { ... },
  "meta": { "page": 1, "total": 45, "pages": 5 }
}
```

7.1 Auth

```
POST   /auth/login/
POST   /auth/refresh/
POST   /auth/logout/
GET    /auth/me/
PATCH  /auth/me/pin/         → change billing PIN
```

7.2 Products

```
GET    /products/?search=&schedule=&drugType=&outletId=
POST   /products/                    → create master product
GET    /products/:id/
PATCH  /products/:id/
GET    /products/:id/batches/        → all batches for product
GET    /products/search/?q=          → for billing autocomplete
                                returns: name, batches with qty+expiry
```

7.3 Inventory (Batches)

```
GET    /inventory/?outletId=&filter=all|low|expiring|out_of_stock
       Returns: products with aggregated batch data
       Query params: search, schedule, drugType, page, pageSize
GET    /inventory/:productId/batches/     → all batches for a product
POST   /inventory/adjust/
       Body: { batchId, type: 'damage'|'return'|'correction', qty, reason, pin }
GET    /inventory/alerts/
       Returns: { lowStock: [...], expiringIn30Days: [...], outOfStock: [...] }
GET    /inventory/export/csv/        → download full stock sheet
```

7.4 Distributors

```
GET    /distributors/?outletId=
POST   /distributors/
GET    /distributors/:id/
PATCH  /distributors/:id/
DELETE /distributors/:id/            → soft delete (isActive=false)
GET    /distributors/:id/ledger/?from=&to= → LedgerEntry[]
GET    /distributors/:id/outstanding/     → unpaid invoice list
```

7.5 Purchases

```
GET    /purchases/?outletId=&status=&from=&to=&distributorId=&search=&page=
POST   /purchases/
       Body: CreatePurchasePayload (see Section 8)
       Effect:
         1. Create PurchaseInvoice
         2. For each item → Create PurchaseItem
         3. For each item → Create/Update Batch (qtyStrips += qty + freeQty)
         4. If isCustomProduct → Create Product master
```

```
    5. Create LedgerEntry (debit on distributor)
    6. Calculate dueDate = invoiceDate + distributor.creditDays
GET    /purchases/:id/
PATCH  /purchases/:id/                  → update header only (not items)
```

7.6 Sales / Billing

```
GET    /sales/?outletId=&from=&to=&customerId=&page=
POST   /sales/
    Body: CreateSalePayload (see Section 8)
    Effect:
    1. Create SaleInvoice
    2. For each item:
       → Validate batch has enough qty
       → batch.qtyStrips -= qty
       → Create SaleItem
    3. If Schedule H/H1/Narcotic → Create ScheduleHRegister entry
    4. If credit sale → customer.outstandingBalance += grandTotal
    5. Create LedgerEntry if credit customer
    6. Auto-generate invoiceNo (sequential per outlet)
GET    /sales/:id/
GET    /sales/:id/print/                → PDF-ready response
GET    /sales/invoice-number/           → next invoice number preview
```

7.7 Customers

```
GET    /customers/?search=&outletId=&hasOutstanding=
POST   /customers/
GET    /customers/:id/
PATCH  /customers/:id/
GET    /customers/:id/purchase-history/
GET    /customers/:id/ledger/?from=&to=
GET    /customers/:id/chronic-medicines/
```

7.8 Accounts / Payments

```
-- Distributor Payments
GET    /payments/?distributorId=&from=&to=
POST   /payments/
    Body: { distributorId, date, totalAmount, paymentMode,
            referenceNo, notes,
            allocations: [{ purchaseInvoiceId, allocatedAmount }] }
```

```
    Effect (in one DB transaction):
      1. Create PaymentEntry
      2. Create PaymentAllocation rows
      3. For each allocation:
        → PurchaseInvoice.amountPaid += allocatedAmount
        → PurchaseInvoice.outstanding -= allocatedAmount
      4. Create LedgerEntry (credit on distributor ledger)

-- Customer Receipts
GET    /receipts/?customerId=&from=&to=
POST   /receipts/
    Same pattern as payments but for customers/sale invoices

-- Outstanding summaries
GET    /outstanding/distributors/
    Returns: DistributorOutstanding[] sorted by overdueAmount DESC
GET    /outstanding/customers/
    Returns: CustomerOutstanding[] sorted by overdueAmount DESC

-- Expenses
GET    /expenses/?from=&to=&head=
POST   /expenses/
    Body: { date, expenseHead, customHead, amount, paymentMode, referenceNo, notes }
```

7.9 Staff

```
GET    /staff/?outletId=
POST   /staff/
GET    /staff/:id/
PATCH  /staff/:id/
DELETE /staff/:id/                    → soft delete
GET    /staff/:id/attendance/?month=2026-03
GET    /staff/:id/performance/        → billing stats
POST   /staff/:id/verify-pin/         → { pin } → { valid: true/false }
```

7.10 Attendance

```
POST   /attendance/mark/
    Body: { staffId, type: 'check_in'|'check_out', photo: File }
    Effect: Upload photo to R2, create AttendanceRecord
GET    /attendance/?outletId=&date=&month=
GET    /attendance/today/             → all staff status for today
```

7.11 Reports

```
GET   /reports/sales/daily/?date=
GET   /reports/sales/summary/?from=&to=
GET   /reports/gst/gstr1/?from=&to=      → GSTR-1 format
GET   /reports/gst/gstr2/?from=&to=      → GSTR-2 format (purchases)
GET   /reports/gst/gstr3b/?month=        → GSTR-3B summary
GET   /reports/inventory/valuation/
GET   /reports/inventory/movement/?productId=&from=&to=
GET   /reports/expiry/?days=30
GET   /reports/staff/performance/?from=&to=
```

8. KEY REQUEST/RESPONSE SHAPES

CreatePurchasePayload

```
{
  "distributorId": "uuid",
  "purchaseType": "credit",
  "invoiceNo": "AJD-2026-0089",
  "invoiceDate": "2026-03-15",
  "godown": "main",
  "notes": "optional",
  "items": [
    {
      "productId": "uuid-or-null",
      "isCustomProduct": false,
      "customProductName": null,
      "batchNo": "AJ2025M1",
      "expiryDate": "2027-03-31",
      "qty": 100,
      "freeQty": 10,
      "purchaseRate": 45.00,
      "ptr": 52.00,
      "pts": 50.00,
      "mrp": 65.00,
      "gstRate": 12,
      "cessRate": 0,
      "discountPct": 5
    }
  ]
}
```

CreateSalePayload

```
{
  "customerId": "uuid-or-null",
  "customerName": "Walk-in",
  "paymentMode": "split",
  "cashAmount": 200,
  "upiAmount": 204,
  "upiRef": "UPI123456",
  "items": [
    {
      "productId": "uuid",
      "batchId": "uuid",
      "qty": 2,
      "mrp": 65.00,
      "saleRate": 61.75,
      "gstRate": 12,
      "discountPct": 5
    }
  ],
  "scheduleHEntries": [
    {
      "productId": "uuid",
      "saleItemIndex": 0,
      "doctorName": "Dr. Sharma",
      "patientName": "Ramesh Kumar",
      "patientAge": 45,
      "prescriptionNo": "RX-001"
    }
  ]
}
```

9. CRITICAL BUSINESS LOGIC

The backend must implement these correctly — wrong logic here directly impacts the pharmacist's money.

9.1 Batch Stock Update on Purchase Save

```
# In one atomic transaction:
def save_purchase(payload):
    invoice = PurchaseInvoice.create(...)
```

```
    for item in payload.items:
        if item.isCustomProduct:
            product = Product.create(name=item.customProductName)

        purchase_item = PurchaseItem.create(invoiceId=invoice.id, ...)

        # Create or update batch
        batch, created = Batch.get_or_create(
            outletId=invoice.outletId,
            productId=product.id,
            batchNo=item.batchNo,
            expiryDate=item.expiryDate
        )
        batch.qtyStrips += item.qty + item.freeQty  # freeQty adds to stock
        batch.save()

    # Distributor ledger
    running_balance = get_last_balance(distributorId) + invoice.grandTotal
    LedgerEntry.create(
        entityType='distributor',
        entityId=invoice.distributorId,
        entryType='purchase',
        debit=invoice.grandTotal,
        balance=running_balance
    )

    # Calculate due date
    invoice.dueDate = invoice.invoiceDate + timedelta(days=distributor.creditDays)
```

9.2 Batch Selection on Sale (FEFO)

```
# First Expiry First Out — legal requirement for pharmacy
def get_batch_for_sale(productId, outletId, qtyRequired):
    batches = Batch.objects.filter(
        productId=productId,
        outletId=outletId,
        qtyStrips__gt=0,
        expiryDate__gt=date.today()
    ).order_by('expiryDate')  # earliest expiry first

    if sum(b.qtyStrips for b in batches) < qtyRequired:
        raise InsufficientStockError
```

```
    return batches  # frontend picks from this list
```

9.3 Bill-by-Bill Payment (most complex)

```
# In one atomic transaction:
def record_payment(payload):
    total_allocated = sum(a.allocatedAmount for a in payload.allocations)
    assert total_allocated == payload.totalAmount  # must match

    payment = PaymentEntry.create(...)

    for allocation in payload.allocations:
        invoice = PurchaseInvoice.get(allocation.purchaseInvoiceId)
        assert allocation.allocatedAmount <= invoice.outstanding

        PaymentAllocation.create(
            paymentEntryId=payment.id,
            purchaseInvoiceId=invoice.id,
            allocatedAmount=allocation.allocatedAmount
        )
        invoice.amountPaid += allocation.allocatedAmount
        invoice.outstanding -= allocation.allocatedAmount
        invoice.save()

    # Ledger entry (credit = money going to distributor)
    running_balance = get_last_balance(distributorId) - payment.totalAmount
    LedgerEntry.create(
        entityType='distributor',
        entityId=payment.distributorId,
        entryType='payment',
        credit=payment.totalAmount,
        balance=running_balance  # reduces what you owe
    )
```

9.4 Auto Invoice Numbering

```
def generate_invoice_no(outletId):
    year = date.today().year
    last = SaleInvoice.objects.filter(
        outletId=outletId,
        invoiceNo__startswith=f'INV-{year}-'
    ).order_by('-createdAt').first()
```

```
    if last:
        num = int(last.invoiceNo.split('-')[-1]) + 1
    else:
        num = 1

    return f'INV-{year}-{str(num).zfill(6)}'  # INV-2026-000001
```

9.5 Schedule H Blocking

```
def validate_sale_items(items):
    for item in items:
        product = Product.get(item.productId)
        if product.schedule in ['H', 'H1', 'X', 'Narcotic']:
            if not item.scheduleHEntry:
                raise ValidationError(
                    f"{product.name} is Schedule {product.schedule}. "
                    "Doctor and patient details are required."
                )
```

10. WHAT IS CURRENTLY BUILT (FRONTEND — 16 STAGES)

All screens are complete with mock data. When USE_MOCK=false in .env, these hooks hit the real API:

Stage	Module	Key Hooks	API Needed
1-2	Infrastructure + Types	—	Auth endpoints
3-4	Dashboard + Nav	—	/reports/sales/daily/
5	Product Management	useProducts	/products/
6-7	Billing POS + Payment	useBilling, useCreateSale	/sales/, /products/search/
8	Inventory	useInventory, useAdjustStock	/inventory/
9	Purchases + GRN	usePurchasesList, useCreatePurchase	/purchases/, /distributors/
10	Credit / Udhari	useCreditAccounts	/customers/:id/ledger/
11	Customers	useCustomers, useCustomerHistory	/customers/
12	Staff	useStaff	/staff/
13	Attendance	useAttendance	/attendance/
14	Reports	useReports	/reports/*

15	Settings	useSettings	/outlet/settings/
16	Accounts	useAccounts, usePayments	/payments/, /receipts/, /expenses/, /outstanding/

11. HOW TO CONNECT THE REAL API

The frontend uses a single toggle. In .env:
USE_MOCK=false # flip this
NEXT_PUBLIC_API_URL=https://api.mediflow.in/api/v1

In lib/api.ts, every hook's mutationFn / queryFn switches from localStorage mock to real Axios call. Zero component changes needed. The swap happens module by module — you can turn on auth + products first while billing is still mocked. Each module is independently toggleable.

12. PHASE 1 LAUNCH CHECKLIST

These are the only endpoints needed for the pharmacy to go live. Everything else can come in Phase 2:

- ✅ **Auth** (login/logout/me)
- ✅ **Products** (search for billing)
- ✅ **Batches** (stock for billing)
- ✅ **Sales** — create + print invoice
- ✅ **Purchases** — create GRN + batch update
- ✅ **Distributors** — CRUD
- ✅ **Customers** — basic CRUD
- ✅ **Outstanding** — distributor totals
- ✅ **Payments** — record payment against invoice
- ✅ **Reports** — daily sales + GSTR-1/2/3B export
- ✅ **Attendance** — check-in/check-out with photo

Phase 2 (post-launch):

- Receipts (customer payments)
- Expense tracking
- Full ledger drill-down
- Debit/Credit notes
- P&L, Balance Sheet
- WhatsApp integration
- Marg data migration scripts

13. ENVIRONMENT & DEPLOYMENT

# .env.backend
SECRET_KEY=<django-secret>
DEBUG=false

```
ALLOWED_HOSTS=api.mediflow.in
DATABASE_URL=postgresql://user:pass@db:5432/mediflow
REDIS_URL=redis://redis:6379/0
R2_ACCOUNT_ID=<cloudflare>
R2_ACCESS_KEY=<key>
R2_SECRET_KEY=<secret>
R2_BUCKET_NAME=mediflow-files
CORS_ALLOWED_ORIGINS=[https://mediflow.in](https://mediflow.in)

# Run
docker-compose up -d
python manage.py migrate
python manage.py createsuperuser
python manage.py seed_demo_data   # seeds 1 outlet + products + mock stock
```

14. IMPORTANT RULES FOR BACKEND DEVELOPER

- 🚨 **Every query must filter by outletId** — never return cross-outlet data under any circumstance.
- 🚨 **Purchase save must be atomic** — all 5 steps in one DB transaction or none.
- 🚨 **Payment recording must be atomic** — PaymentEntry + allocations + invoice updates together.
- 🚨 **Never manually insert LedgerEntry** — always auto-create from the service layer, never from views.
- 🚨 **Batch stock can never go below 0** — raise InsufficientStockError before billing.
- 🚨 **outstanding on invoices is stored, not computed** — update it on every payment, don't recalculate from scratch.
- 🚨 **Invoice numbers are sequential per outlet per year** — use SELECT FOR UPDATE to prevent race conditions.
- 🚨 **Schedule H items always create ScheduleHRegister entries** — no exceptions, it is a legal requirement under Indian drug law.